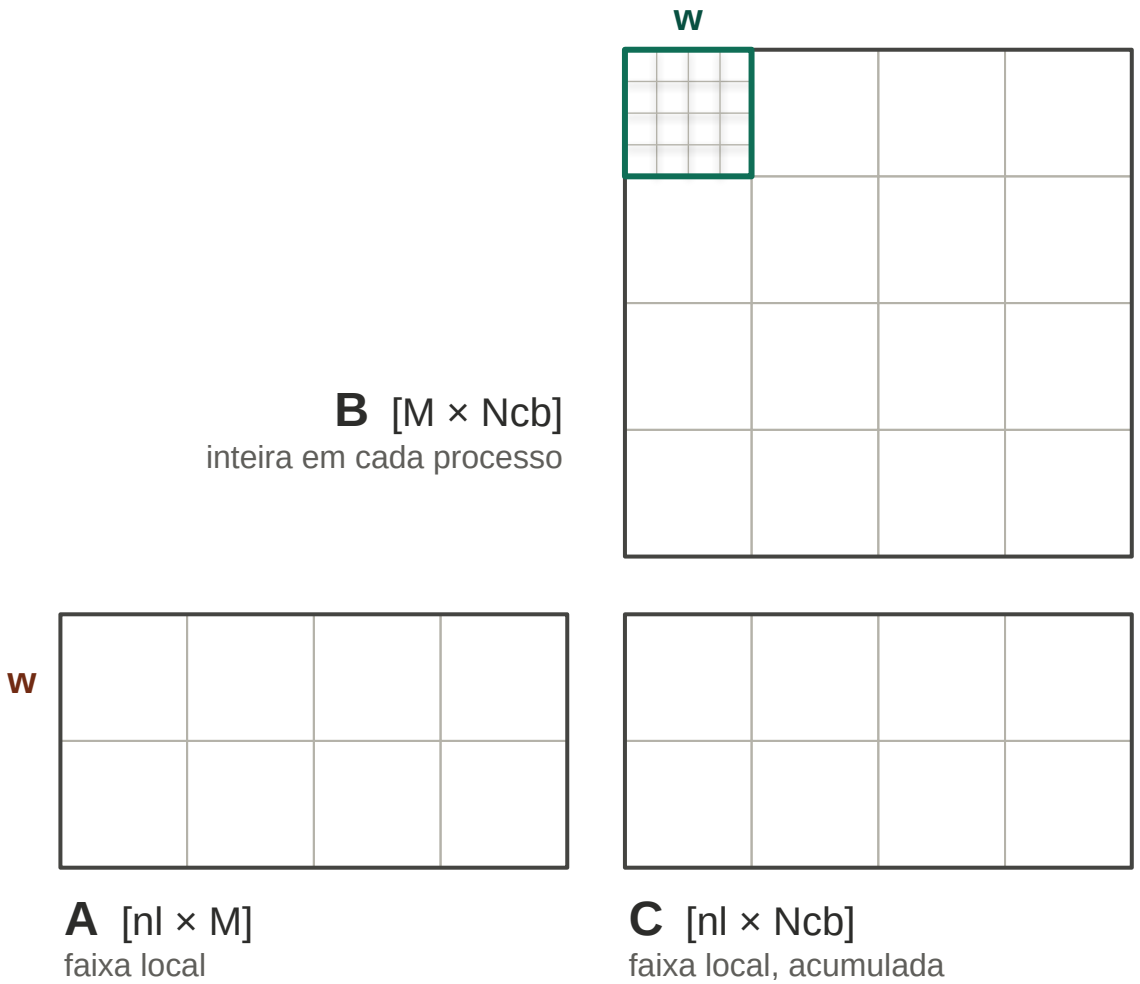
Multiplicação de matrizes blocada com ladrilhos

C = A × B · versão blocada (tiling) do kernel local em MPI



As matrizes divididas em ladrilhos w × w

Cada processo guarda B inteira e a sua faixa de nl linhas de A e de C



Ladrilho (bloco): um quadrado de w × w elementos.

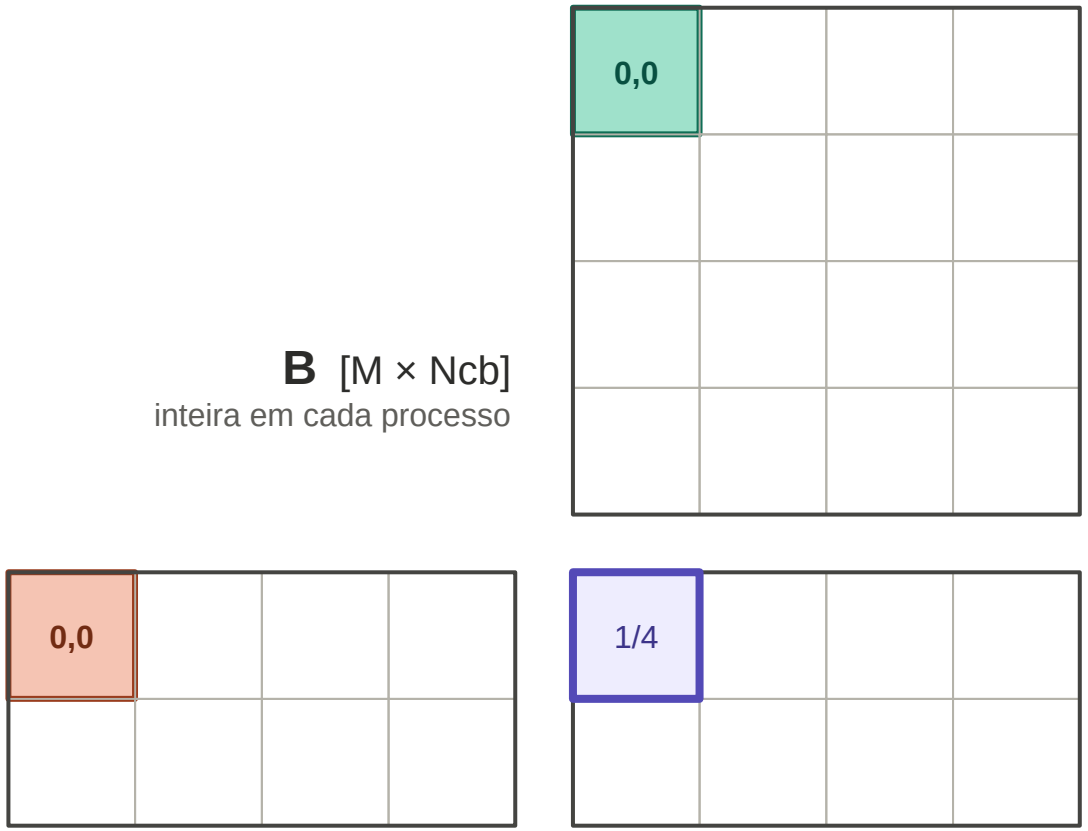
Índices dos blocos: ii (linhas de A e C), kk (colunas de A = linhas de B), jj (colunas de B e C).

Restrições do exercício: w múltiplo de 8 e divisor de M e de Ncb. Só o último bloco de linhas da faixa (nl) pode ser menor.

Por que blocos? Enquanto um bloco de B está no cache, ele é reaproveitado por todas as linhas do bloco de A, em vez de ser relido da memória.

Ordem dos laços externos: ii → kk → jj

Passo 1 de 32 (nl = 2w, M = Ncb = 4w)




```
ii = 0   kk = 0   jj = 0

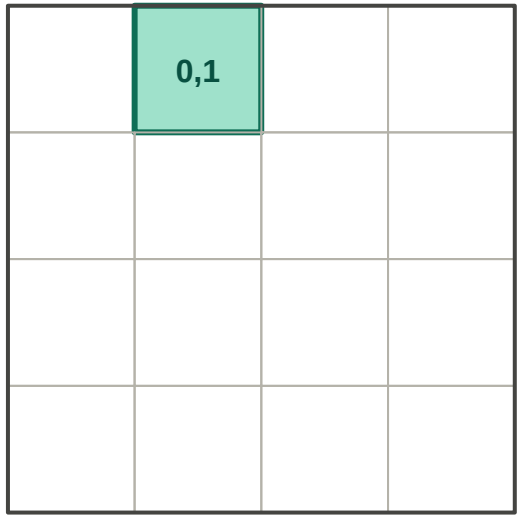
C[0][0] += A[0][0] · B[0][0]
```

Primeiro passo: três blocos w × w estão em uso, um de cada matriz.

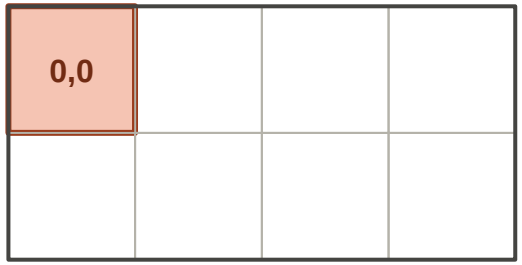
- bloco de A em uso
- bloco de B em uso
- bloco de C atualizado
- contribuições já somadas

Ordem dos laços externos: ii → kk → jj

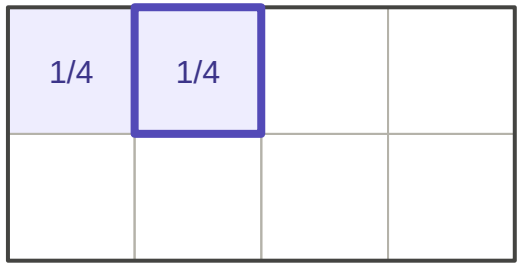
Passo 2 de 32 (nl = 2w, M = Ncb = 4w)



B [M × Ncb]
inteira em cada processo



A [nl × M]
faixa local



C [nl × Ncb]
faixa local, acumulada

```
ii = 0   kk = 0   jj = 1
```

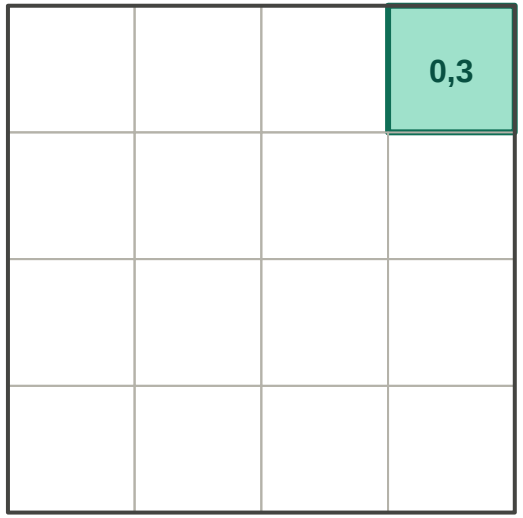
```
C[0][1] += A[0][0] · B[0][1]
```

O bloco A[0][0] continua no cache; só o bloco de B muda.

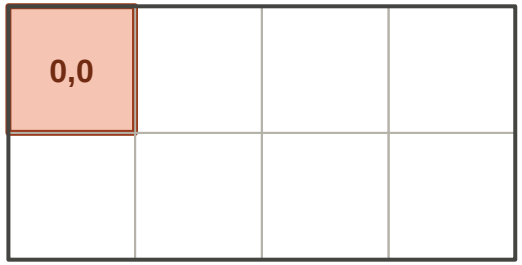
- bloco de A em uso
- bloco de B em uso
- bloco de C atualizado
- contribuições já somadas

Ordem dos laços externos: ii → kk → jj

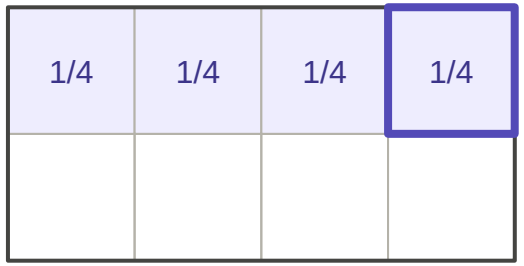
Passo 4 de 32 (nl = 2w, M = Ncb = 4w)



B [M × Ncb]
inteira em cada processo



A [nl × M]
faixa local



C [nl × Ncb]
faixa local, acumulada

`ii = 0   kk = 0   jj = 3`

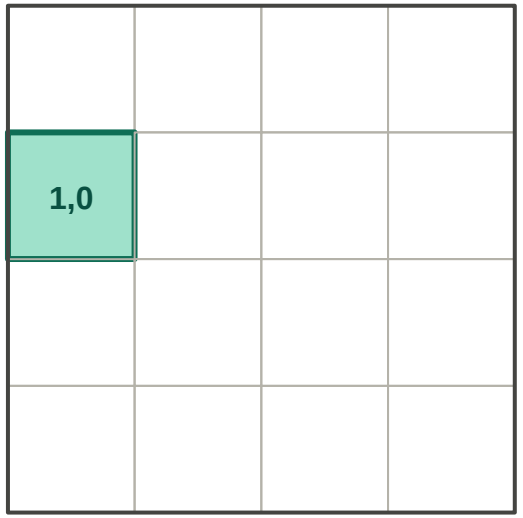
`C[0][3] += A[0][0] · B[0][3]`

Fim da varredura em jj: A[0][0] foi usado com toda a linha de blocos de B.

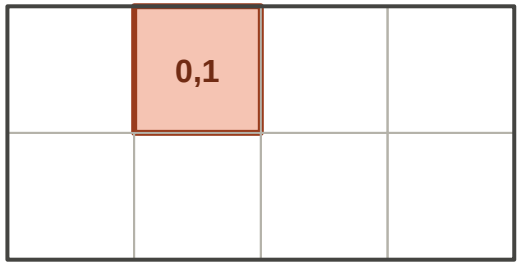
- bloco de A em uso
- bloco de B em uso
- bloco de C atualizado
- contribuições já somadas

Ordem dos laços externos: ii → kk → jj

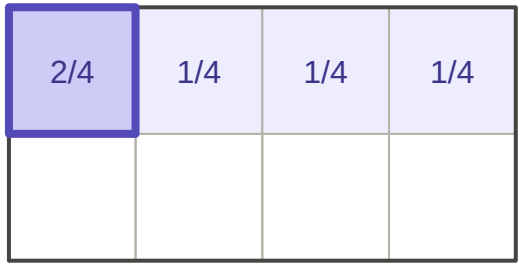
Passo 5 de 32 (nl = 2w, M = Ncb = 4w)



B [M × Ncb]
inteira em cada processo



A [nl × M]
faixa local



C [nl × Ncb]
faixa local, acumulada

```
ii = 0   kk = 1   jj = 0

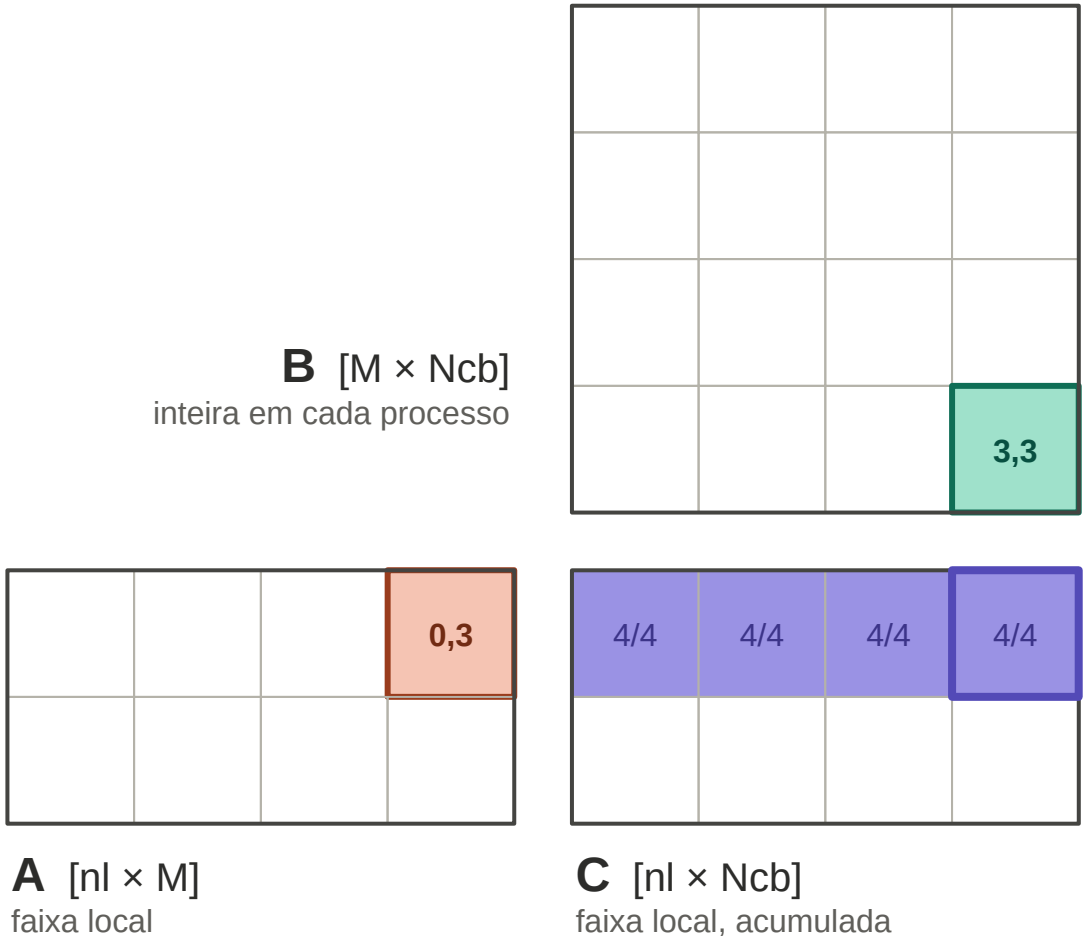
C[0][0] += A[0][1] · B[1][0]
```

kk avança: novo bloco de A e a próxima linha de blocos de B. Cada bloco de C recebe mais uma contribuição.

- bloco de A em uso
- bloco de B em uso
- bloco de C atualizado
- contribuições já somadas

Ordem dos laços externos: ii → kk → jj

Passo 16 de 32 (nl = 2w, M = Ncb = 4w)



`ii = 0   kk = 3   jj = 3`

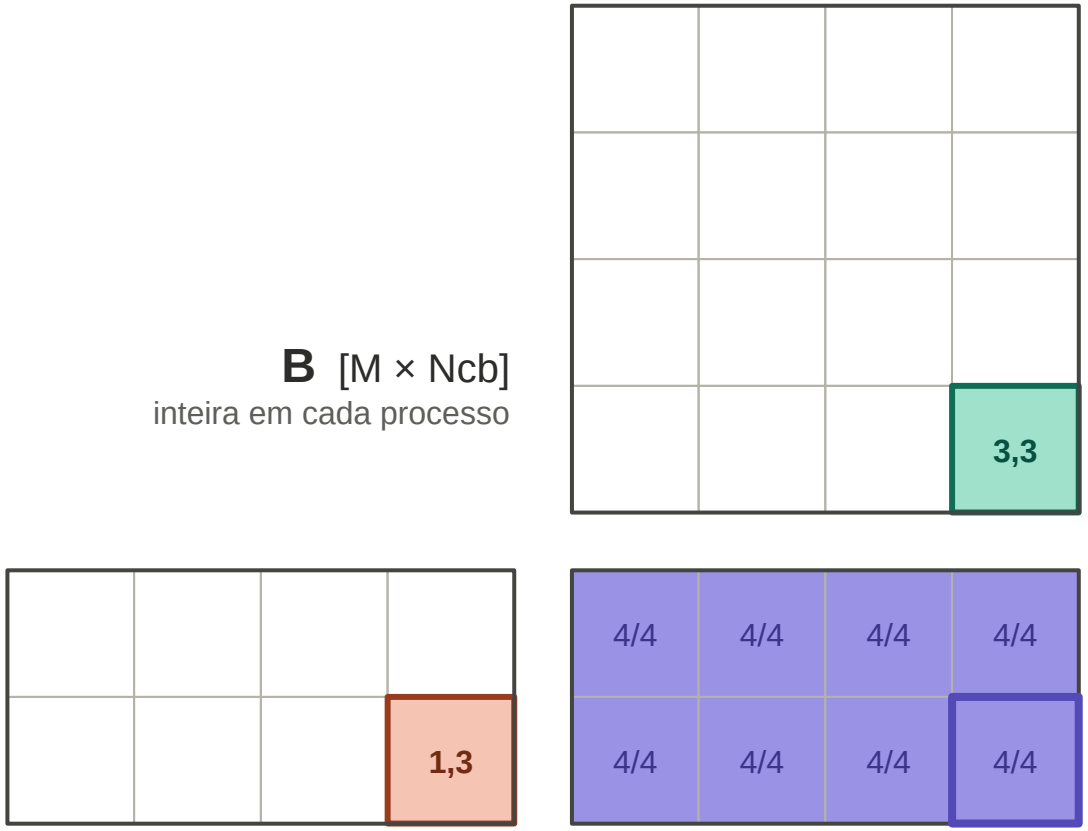
`C[0][3] += A[0][3] · B[3][3]`

Linha de blocos ii = 0 de C concluída: cada bloco recebeu as M/w = 4 contribuições.

- bloco de A em uso
- bloco de B em uso
- bloco de C atualizado
- contribuições já somadas

Ordem dos laços externos: ii → kk → jj

Passo 32 de 32 (nl = 2w, M = Ncb = 4w)



B [M × Ncb]
inteira em cada processo

A [nl × M]
faixa local

C [nl × Ncb]
faixa local, acumulada

`ii = 1   kk = 3   jj = 3`

`C[1][3] += A[1][3] · B[3][3]`

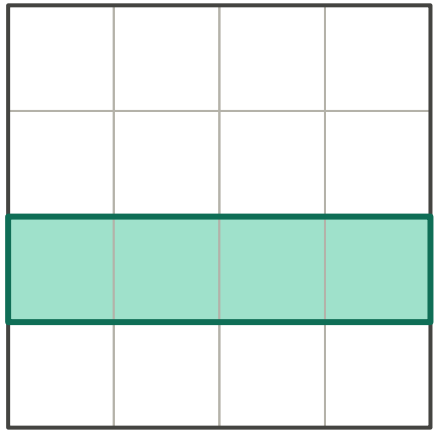
Último passo: a faixa local de C está completa

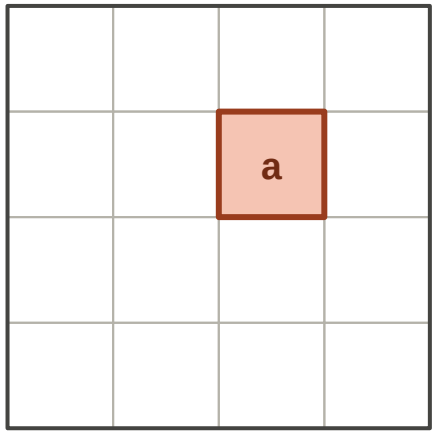
- bloco de A em uso
- bloco de B em uso
- bloco de C atualizado
- contribuições já somadas

A multiplicaçao dentro de um ladrilho: laços internos i → k → j

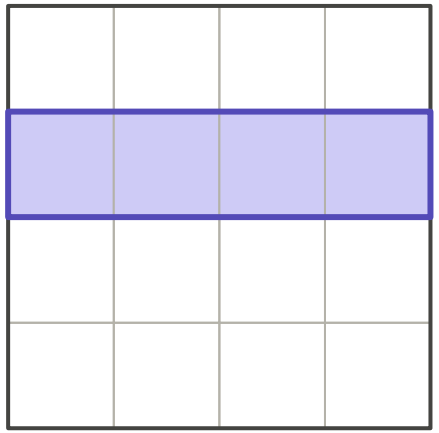
Ampliação de um passo: um bloco de A, um de B e um de C, com w = 4 para o desenho

linha k de B
lida em sequência





`a = A[i][k]`
um escalar



linha i de C
acumulada com += a·b[j]

```
for (i = ii; i < ie; i++)
  for (k = kk; k < kk + w; k++) {
    a = A[i][k];
    for (j = jj; j < jj + w; j++)
      C[i][j] += a * B[k][j];
  }
```

Laço interno em j: percorre uma linha de B e uma de C em posições consecutivas da memória.

Reuso: cada linha k do bloco de B é usada uma vez para cada uma das w linhas i. Com o bloco no cache, cada elemento de B trazido da memória serve w vezes.